



Qatar University

College of Engineering

Department of Computer Science and Engineering

Internship Report At Applab

Designing and Implementing AI Solutions with Azure

[Redacted]

Mentor: [Redacted]

2025

This project report is submitted to the Department of Computer Science and Engineering of Qatar University in partial fulfillment of the requirements of the Practical Training course.

Abstract

This report outlines my internship experience as a Software and AI Engineer at Applab. My main goal was to connect what I learned in theory with real-life by helping to create practical AI solutions. I worked on developing a "Bring Your Own Data" chatbot, improving an existing chatbot, and creating a real-time desktop translation application. My key achievements involved architecting the chatbot from scratch using a Retrieval Augmented Generation (RAG) pattern with Azure OpenAI and AI Search. More importantly, I identified and fixed the major issues of the desktop translator by redesigning its audio pipeline. This work removed feedback loops and addressed conflicts with the digital audio sample rate. The primary learning outcomes from these experiences include a deep, practical proficiency in the Microsoft Azure ecosystem and the development of crucial professional skills in independent problem-solving, project development, and client communication.

Acknowledgment

I would like to express gratitude to my supervisor, Tom Saji, for his guidance and support. I would also like to thank my friends for their constant encouragement throughout this internship period.

Table of Contents

Abstract ii

Acknowledgment ii

1. Background of the organization4

2. Internship learning objectives4

 2.1. Activity Objectives4

 2.2. Growth Objectives4

3. Internship Experience.....5

 3.1. Learning Azure Fundamentals5

 3.2. Bring Your Own Data (BYOD) Chatbot5

 3.3. Manateq Chatbot Automation7

 3.4. Qrail Desktop Meeting Translator9

4. Reflections and learning outcomes11

5. References.....12

1. Background of the organization

Applab is a Qatari technology company that specializes in providing custom software solutions for businesses, such as developing websites and mobile apps. They also offer services in security, automation, and data analytics to help their clients work smarter and make better decisions.

At Applab, I was given the opportunity to intern as an Associate Software & AI Engineer within the Emerging Technologies (AI and Data) Department. The Emerging Technologies Department specializes in developing solutions that integrate the latest advancements in software and artificial intelligence. The department consists of a multidisciplinary team of AI Engineers, Software Developers, Cloud Architects, and DevOps Engineers, etc, who work collaboratively on projects to address real-world business needs and challenges.

The department mainly uses Microsoft Azure due to its wide range of services for AI, cloud computing, and application hosting. In addition to Azure, the team also integrates other technologies to support the various development needs of the clients such as Python, .NET, FastAPI for developing APIs, Docker for containerizing applications etc.

2. Internship learning objectives

At the beginning of my internship at Applab, I set several activity and growth objectives along with my supervisor to create a clear roadmap to ensure I would gain actual technical skills and professional experience. Throughout the internship, I achieved many of these objectives by moving from foundational learning to hands-on application through real-world projects.

2.1. Activity Objectives

- To become familiar with the core concepts of cloud computing and Azure AI services.
- To learn how to integrate AI models into functional software systems.
- To gain hands-on experience developing cloud-based software components as assigned.
- To acquire real-world practical experience relevant to my future career.
- To actively support the team by assisting with implementation, debugging, or other tasks.
- To explore new professional tools and document my findings and progress.
- To participate in team discussions and meetings to understand how project decisions are made.

2.2. Growth Objectives

- To improve my problem-solving skills in a professional context.
- To build confidence working with new and unfamiliar technologies.
- To learn how to manage tasks and deadlines effectively.
- To enhance my ability to seek out knowledge independently using professional resources.
- To develop my communication skills by sharing updates and asking relevant questions.
- To learn when it is appropriate to seek help and guidance from supervisors and colleagues.

These objectives represented a direct link between my university courses and the real-world projects at my internship. My goal was to move from just understanding theory to actually using it.

These practical goals helped me develop my skills and prepared me to contribute in an important way to the client projects which are at the core of Applab's operations.

3. Internship Experience

My internship at Applab was a journey that began with foundational self-learning and progressed to direct involvement in complex, real-world client projects.

3.1. Learning Azure Fundamentals

The first two weeks of my internship started out with conceptual learning. I started by familiarizing myself with the basics of cloud computing and Microsoft Azure. This included building a strong foundation in cloud service types such as IaaS, PaaS, and SaaS, cloud models, global infrastructure components along with essential security principles.

Building on this, I delved into the AI-102 (Designing and Implementing a Microsoft Azure AI Solutions) curriculum. I chose this path as it allowed me to gain a comprehensive understanding of the Azure AI domain, which covered a wide range of topics such as developing Generative AI Solutions, Agentic Solutions, Natural Language Processing (NLP), Computer Vision, etc. I believe that these two weeks were the most fundamental during my internship, as the time I invested in the modules greatly simplified the hands-on work for me later. While the entire curriculum was useful, a few key areas were at the core of the projects I was assigned:

- **Generative AI and RAG for the BYOD Chatbot:** The module on Developing Gen AI Solutions introduced me to the Retrieval Augmented Generation (RAG) pattern, a technique that allows language models to generate responses based on a custom, private data source that you upload. This was the exact architectural pattern required for my first hands-on task: the "Bring Your Own Data" chatbot. The training taught me how to combine *Azure OpenAI* for its reasoning capabilities with *Azure AI Search* for powerful indexing and retrieval functions. The Microsoft Training labs I followed during this period were essential in building my confidence in starting my own implementation from scratch.
- **Azure AI Speech for the QRail Desktop Translator:** Arguably, the most important part of the training was the modules related to the *Azure AI Speech* service, as it became the core of my work on the QRail project. The goal was to transcribe, translate, and synthesize live meeting audio, and this module provided me with all the necessary knowledge. Specifically, I learned about Speech-to-Text (Speech Recognition), Text-to-Speech (Speech Synthesis), and the Azure AI Speech SDK. The knowledge gained here allowed me to quickly get a working prototype and focus on the more unique challenges of the project, such as audio routing and dealing with specific audio settings like sample rates, etc.

3.2. Bring Your Own Data (BYOD) Chatbot

After completing the Azure AI services modules, my first hands-on assignment was an internal project meant for the supervisor to evaluate my understanding of the modules and assess my ability to design, integrate, and deploy a modern AI solution. The objective was to build a complete "Bring Your Own Data" (BYOD) chatbot entirely from scratch which allows a user to upload their own private

documents and then engage in a natural language conversation to ask questions and retrieve information from that content.

I accomplished this by basing the system on the Retrieval Augmented Generation (RAG) pattern, which ensures relevance and prevents hallucinations by grounding the responses in real, user-provided data. If an answer is not found in the provided data, the chatbot does not make up an answer. For the application's backend, I used FastAPI which allowed me to later scale the application for more features.

The chatbot flow works as follows: A user uploads a document via the frontend or the "upload" endpoint within the FastAPI application. As a result, the raw file is saved in Azure Blob Storage by the backend. Once a document is uploaded, it causes the Azure AI Search indexer to run. An indexer is a component that scans and processes data to prepare it for search. I configured it with a custom skillset which can be defined as a collection of AI-powered steps (or "skills") that are used to enrich or transform the content. In this case, I added a Text Split skill to divide long text into smaller, manageable chunks. Then it used an Azure OpenAI Embedding model to convert each chunk into a numeric vector that represents the meaning of the text. These vectors, along with the original text, are then stored in a searchable vector index.

When a user asked a question, the backend converts the question into a vector using the same embedding model. It then searched the index to find the most relevant parts of the original documents. These parts, along with the question, were sent to an Azure OpenAI model (GPT-4o), which generated a clear and relevant answer. Finally, to maintain a stateful and continuous conversation, this interaction pair (the user's query and the chatbot's response) is written as a new document to an Azure Cosmos DB database, providing a complete chat history for each session. This flow, inspired by [\[1\]](#), can be seen as shown in the figure below. I also used Docker to containerize the whole application, which makes it easy to run and deploy anywhere.

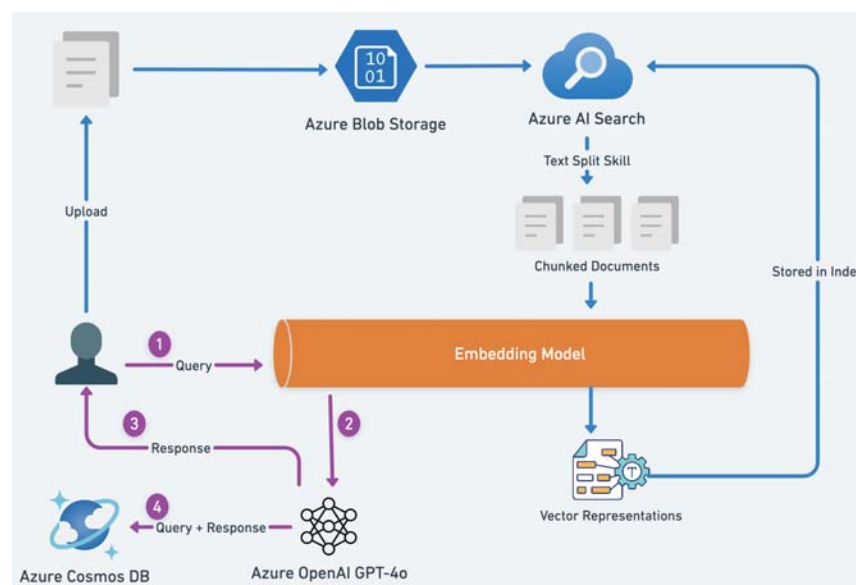


Figure 1 - RAG Architecture Diagram for Chatbot

Although not a part of my tasks, I also developed a frontend interface using Next.js and Tailwind CSS to not only remain productive during other project delays but also as a chance to refresh my development skills from a previous internship. This provided me with a functional way to interact with the backend API.

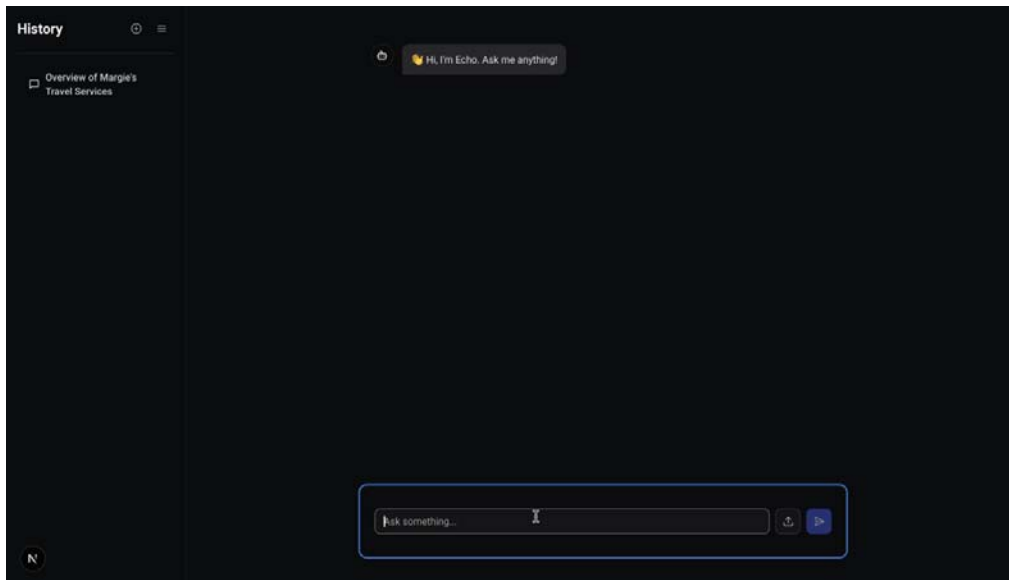


Figure 2 - Frontend UI for Chatbot

Technical Challenges

- One of the biggest challenges I faced was getting the Dockerfile to work for the FastAPI app. Docker was completely new to me, so I faced issues with structuring the file correctly. I ran into several errors and had to troubleshoot a lot before I got it working properly. I achieved this by looking at other Dockerfile templates available online and going through the documentation.
- Another big challenge was implementing the chat history feature using Azure Cosmos DB. This service wasn't covered in the training modules I had completed so it was overwhelming at first. To overcome this, I started self-learning by reading Microsoft's documentation and modules. Eventually, I was able to learn the basics of NoSQL data modeling, design a simple schema for storing conversations, and successfully integrate Cosmos DB into my project.

Non-Technical Challenges

- A non-technical challenge I faced at the beginning of the project was feeling unsure about whether I was headed in the right direction. I was confused as to whether I was approaching a task the right way, especially when there wasn't a clear guide/requirements set. I overcame this by actively asking questions to ensure clarity.

3.3. Manateq Chatbot Automation

After successfully demonstrating my knowledge with the BYOD chatbot, I was onboarded to contribute to an active client project for Manateq. The goal of this project was to enhance their existing chatbot. My task was to fix an inefficiency in how the chatbot's knowledge base was maintained.

The current process was manual as whenever the Manateq website was updated, new data had to be manually downloaded, processed, and then uploaded to Azure Blob Storage for the chatbot to reference. This was time-consuming, prone to human error, and created a delay between the website being updated and the chatbot being able to provide information based on that new content.

My task was to design and implement a fully automated solution to replace this manual workflow by using a timer-triggered Azure Function App, which executes the code on a predefined schedule. Due to delays, I was not given access to the function app so I implemented it as a Python script that which can easily be converted to the Function App later. The script works as follows: it calls the Manateq website's internal APIs, gets the latest content in JSON, processes the data to ensure that it is in the correct format for the system and then uploads the files directly to the Azure Blob Storage container ensuring that manual intervention is no longer required.

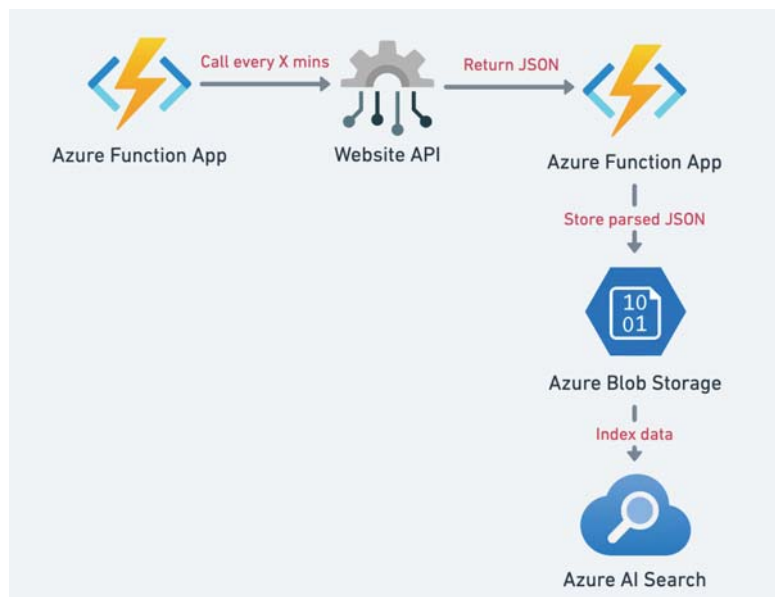


Figure 3 - Architecture Diagram for Function App

Once this task of extracting relevant fields from the JSON was completed, another one of my responsibilities also involved testing the end result. To do this, I connected to the development resources and created another Python script which connected to the chatbot's test API with my newly created index. This allowed me to rigorously query the chatbot to confirm whether it was correctly referencing the newly updated data from the website.

Technical Challenges

- One technical challenge I faced during this project was ensuring that the data retrieved from API was formatted correctly for the chatbot's RAG system. The raw JSON from the API was not structured in a way the chatbot could directly use, and initially the chatbot was failing to retrieve relevant information. To resolve this, I restructured some specific JSON fields into clean, consistent documents before uploading it to Blob Storage.

Non-Technical Challenges

- A non-technical challenge was remaining productive when the project was impacted due to delays in client communication meaning that my access to the resources was postponed. To

stay productive, I utilized this time to shift my focus and contribute to other ongoing projects, specifically by working on the frontend for my chatbot and starting to work on the QRail desktop meeting translator application.

3.4. QRail Desktop Meeting Translator

My final and most rigorous project of the internship was a desktop meeting translator application for QRail. The objective was to create a tool that could provide real-time transcription and translation during live Microsoft Teams meetings. Since an initial draft had been created by another employee, the basic user interface was in place, however, the core functionality was unstable as the software could not reliably capture audio from either the source or destination.

To enable simultaneous and bidirectional translation, the architecture of the application was designed around a multi-threaded model. This was essential for handling both directions of the conversation at once without any delay or interruption and ensuring that the threads continuously capture all audio. The first thread handles the capturing of audio from the physical microphone of the source. It continuously streams that audio to the provisioned Azure Speech service for transcription and translation. It then routes the final synthesized audio out to the virtual cable that Teams is listening to. The second thread is dedicated to listening to the other virtual cable, which captures the audio coming from the meeting participants. It performs the same process in the opposite direction, i.e., sending that audio to Azure for translation into the selected language and then playing the result back to the source.

The first challenge that occurred was severe instability. Based on the audio configuration settings, the application would either fail to capture any audio or, when it partially worked, would immediately create a feedback loop. After a few days, I figured out that this was caused by the use of a single VB-Cable virtual audio device. The initial setup was causing an issue of an infinite loop as a result of the translated audio output being routed from the application back into its own audio input. To fix the issue, I realized I needed to use two distinct VB-Cables to completely isolate the two threads:

1. An Input Cable: This was configured as the speaker output in Microsoft Teams and the microphone input for my Python application. Its sole purpose was to get audio *from* Teams *to* my app.
2. An Output Cable: This was set as the audio output in my application's code and as the user's default listening device (or headphones). Its purpose was to get the final audio *from* my app *to* the user's ears.

This complete separation of the input and output streams immediately stabilized the system and eliminated the feedback loop. Once the core functionality was reliable, I went with the project manager to the QRail offices to present the first demo of the utility. The feedback from this initial demo was positive. However, they provided critical feedback for a new feature so they could hear the original speaker's voice along with the translated voice. This would allow them to confirm who was speaking and maintain a record of meetings for monitoring purposes.

While it sounded really simple, it introduced a much deeper technical challenge. When I enabled both audio streams to play simultaneously, the output of the original voice was extremely distorted. It was challenging to diagnose the issue as it did not occur when handling a single stream, but only

occured later when trying to play both. The two audio paths were logically separate in my code but seemed to be colliding at the final output device.

To solve this, I first had to understand some of the basic terms of digital audio processing and soon realized that the core of the problem was a sample rate conflict. I discovered my code was trying to send two simultaneous audio streams, one at a higher rate and another at a lower rate, to a single audio device which is only capable of processing one at a time. The issue hadn't appeared before because the application previously only played the translated streams sequentially. However, having the audio device handle both, for example, a 44.1 kHz stream and a 16 kHz stream concurrently created a data conflict that resulted in audio distortion.

I made sure that all audio data arrived at the output device in a single, consistent format. This required implementing audio processing and resampling logic directly within the Python application. To visualize and debug this flow, I resorted to sketching out the entire process by hand, as shown in the figure below.

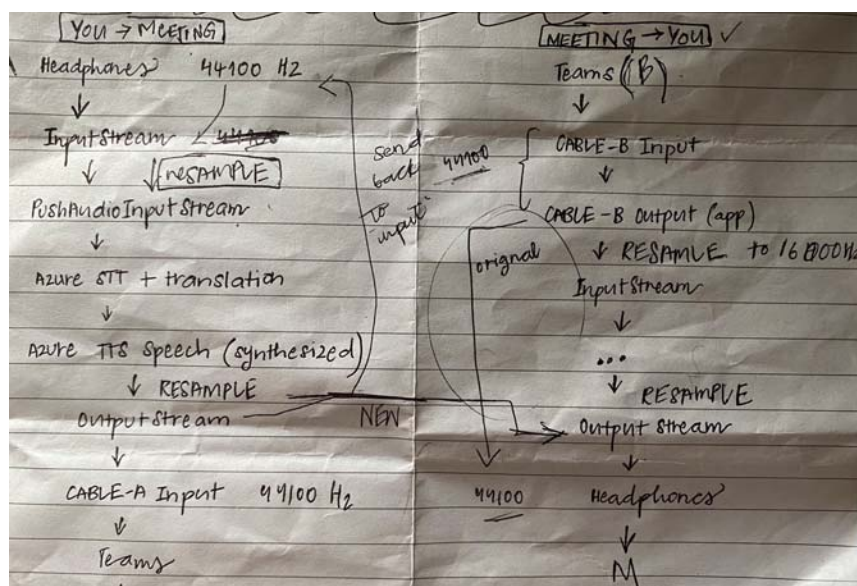


Figure 4 - Debugging flow

A cleaner and more generalized representation of this process can be seen in the architecture diagram below. It showcases the process of one of the threads sending the original and translated audio to the listener.

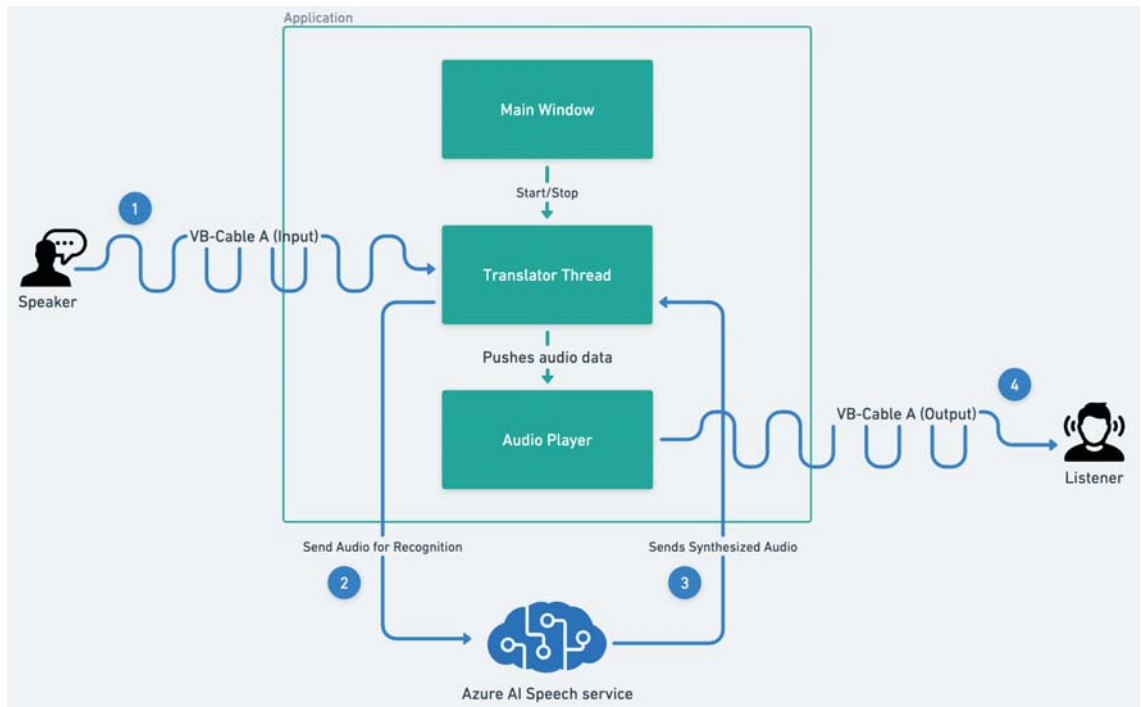


Figure 5 - Architecture Diagram for QRail

With these final issues cleared, we went back to the QRail offices for a final demo. Since they were satisfied, my final task was to transform my script into a distributable product. I used PyInstaller to package the application into standalone executables and installers for both Windows and macOS, and also wrote comprehensive documentation and installation instructions for the client's ease of use.

Technical Challenges

- One of the main challenges I encountered during this project was working with audio concepts and routing, and it initially seemed really overwhelming. I quickly had to learn how to figure out issues related to feedback loops and sample rate mismatches. I fixed these issues by combining both research and hands-on experimentation to ensure the system was stable and reliable.

Non-Technical Challenges

- The most significant non-technical difficulty that I faced during this internship was to work alone to fix and deliver the project all whilst in a tight timeline. Since I didn't have another colleague to share the workload with, I had to manage the pressure of debugging complex issues and developing solutions quickly. To handle this, I addressed each issue step by step. This greatly enhanced my independence, problem-solving skills, and ability to work under pressure.

4. Reflections and learning outcomes

When I started this internship at Applab, I wasn't entirely sure what to expect, so I tried to keep my expectations modest. Looking back, I can say the experience was decent overall, and it certainly exceeded what I had initially hoped for. There were quite a few elements of the internship

where I felt a bit uncertain but it turned out to be a crucial part of the learning process. It challenged me to learn how to adjust, find my own direction, and move forward with my work independently. I'm incredibly happy that I got the opportunity to move beyond theory and learn about the entire lifecycle of an AI solution from design and development to debugging and delivery.

I believe that I met almost all, if not all, of the activity objectives throughout the projects. I met the goal of familiarizing myself with Azure AI services by completing the Azure training modules, which then allowed me to integrate AI models directly into the BYOD chatbot using the RAG pattern. I fulfilled the objective to gain hands-on experience developing cloud-based software by both of the client projects.

My growth objectives were also completed as I participated in bi-weekly stand-up meetings which also helped me in fulfilling my goal to develop my communication skills by improving the way I convey technical progress. The client meetings for the QRail demos pushed this further, reinforcing the importance of clear communication when managing feedback. The unstructured nature of some of the work, which initially felt like a challenge, ultimately helped me enhance my ability to seek out knowledge independently. It forced me to dive into documentation for skills I might not have gained with more rigid, constant mentorship.

This internship also provided me with a lot of insight as to how useful my courses at university were. To mention a few, the Software Engineering course was essential for learning how to plan an implementation, think about non-functional requirements like scalability for the FastAPI backend, and structure a project from start to finish. Another course I found really useful was the Operating Systems course as it provided me with the concepts of threads which allowed the QRail translator app to function in real-time without blocking. Lastly, the concepts I learned during Deep Learning made it a lot easier to understand the modules and implement concepts like document chunking for the chatbot.

This internship has reinforced my passion for problem-solving. While I'm still exploring the exact career path I wish to follow, this experience has given me a much clearer sense of the challenges I find engaging. I'm excited to continue learning and experimenting with different technologies. The tech landscape is evolving at an incredible pace, especially with advancements in AI, and I believe the most important skill is adaptability. My goal is to remain curious, stay open to new opportunities, and continue building skills that allow me to thrive as the world changes.

5. References

[1] "Building a RAG Architecture with Generative AI," *Airbyte*, Aug. 04, 2025.
<https://airbyte.com/data-engineering-resources/rag-architecture-with-generative-ai>